

ANFORDERUNGSANALYSE FÜR ENTWICKLER, TEIL 3

Das Messer richtig ansetzen

Feine Schnitte durch die Anforderungen von der Interaktion bis zur CQS-Funktion.

Ziel der Anforderungsanalyse für Sie als Entwickler ist, Ihnen einen glasklaren Ansatzpunkt für den darauf folgenden Entwurf und dann die Codierung zu geben. Sie müssen nicht nur das Gefühl haben, die Anforderungen zu verstehen, sondern auch die Möglichkeit, den Code zu überprüfen, der daraus entstanden ist. Hat Ihr Gefühl guten Verständnisses Sie getrogen, oder war es berechtigt?

Eine solche Überprüfung kann nur skalierbar mit automatisierten Tests erfolgen. Allein mit ihnen können Sie sowohl zügig die Reife des Codes in ganzer Breite für die aktuelle Anforderung feststellen („Habe ich die Anforderung schon erfüllt?“) wie auch die Stabilität des Codes in Bezug auf alle vorher schon erfüllten Anforderungen („Werden alle anderen Anforderungen immer noch erfüllt?“).

Automatisierte Tests erfordern als Ansatzpunkt eine Funktion und Testfälle. Deshalb bin ich davon überzeugt, dass das Ziel jeder Anforderungsanalyse eine Liste von Funktionen mit präziser Signatur, aufgedeckten Abhängigkeiten von Zustand beziehungsweise Ressourcen und Testfällen sein muss. Wenn Sie also mit einem Product Owner (PO) oder Kunden über Anforderungen sprechen, haben Sie immer im Blick, dass Sie die Frage beantwortet bekommen: „Welche Funktion soll ich implementieren?“

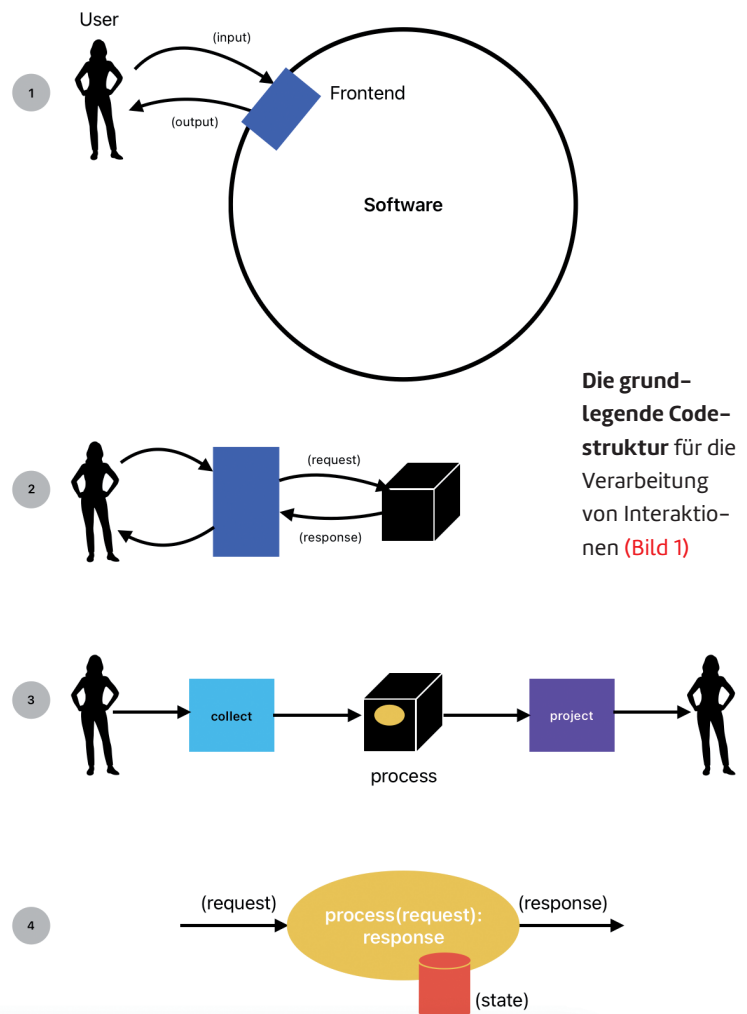
Diese Funktion nenne ich auch Triggerfunktion, denn sie wird von Anwendern der Software im Rahmen ihrer Interaktion durch eine Benutzerschnittstelle angestoßen. Anwender wünschen sich von Software ein gewisses Verhalten. Dazu „reizen“ sie die Software „an der Oberfläche“ und erwarten eine Reaktion: Input-Daten werden in irgendeiner Form eingegeben, eine Funktion wird ausgelöst, Output-Daten werden angezeigt.

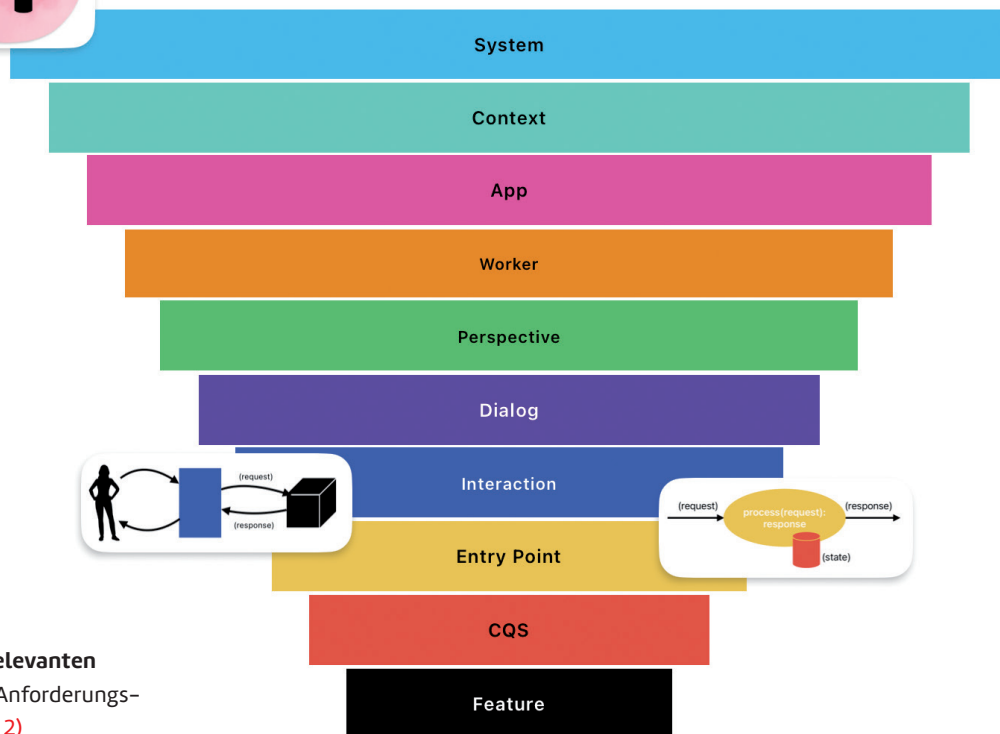
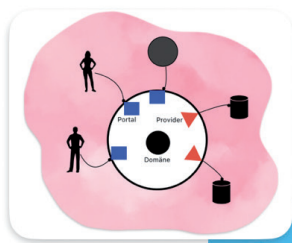
Jede Software ist ganz grundsätzlich in dieser Weise aufgebaut, vergleiche **Bild 1**. Dabei ist es unwesentlich, ob ihre Nutzer Menschen oder andere Software sind.

- User stellen an Software mittels deren Frontend eine Anfrage (Input) und bekommen von der Software ein Ergebnis (Output). Ob das Frontend konsolenbasiert ist, ein GUI hat, eine HTML-Seite ist oder einen REST-Endpoint veröffentlicht, ist einerlei.
- Das Frontend ist der Vermittler zwischen dem User in der Umwelt der Software und „der Maschinerie“ in der Software, die für ihn die wahre Black-box ist. Dort wird das gewünschte Verhalten erzeugt. Das Frontend übersetzt dazu die Anfrage in

einen Request für „die Maschinerie“ und deren Response in etwas Verständliches für den User.

- Jedes Frontend besteht in Bezug auf die Input-Output-Paare, die es akzeptiert, jeweils aus einem Teil, der die Input-Daten vom User sammelt (collect) und in einen Request für die eigentliche Verarbeitung übersetzt, sowie einem Teil, der deren Response zum User hin in verständlicher Weise projiziert (project). Sammlung und Projektion kompensieren die Unfähigkeit beziehungsweise den Unwillen von Usern, direkt mit verarbeitbaren Request-Response-Datenstrukturen umzugehen. Menschliche Benutzer wollen strukturierte Eingabefelder für ihren Input haben und nicht JSON-Datenstrukturen formulieren; sie wollen auch nicht YAML interpretieren, sondern zum Beispiel eine 3D-Grafik mit den produzierten Daten sehen.





Die immer relevanten
Ebenen der Anforderungs-
analyse (Bild 2)

- Der von der Sammlung zusammengestellte Request wird an eine Funktion im Inneren der Software zur Verarbeitung (process) übergeben, die anschließend mit einem Response antwortet. Es kann sein, dass diese Funktion der Black-box auf internen oder externen Zustand (state) lesend und/oder schreibend zugreift.

Wie Sie sich denken können, ist die Funktion, die ich mit Triggerfunktion bezeichne, in Bild 1 *process()*. Wenn sie aufgerufen wird, sind die Feinheiten der Request-Sammlung abgeschlossen, bei der die Frontend-Technologie eine Rolle spielt, die schlecht automatisiert zu testen ist. Und die Feinheiten der Projektion von Ergebnissen für den User mittels Frontend-Technologien, die schlecht zu testen sind, spielen auch für sie keine Rolle; sie sind nachgelagert in der Interaktion.

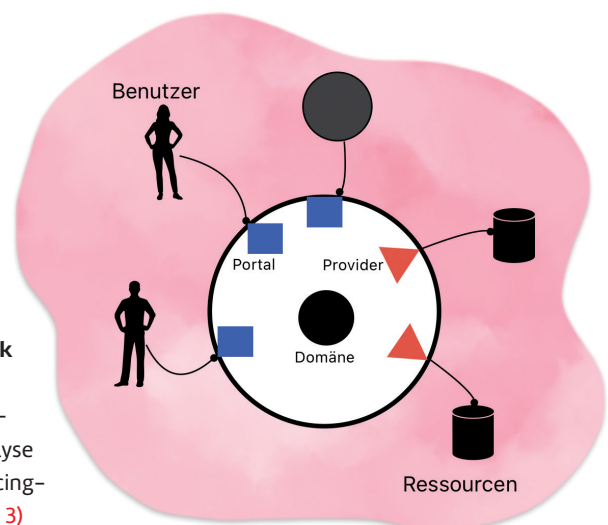
In Bezug auf die im zweiten Teil der Artikelserie [1] vorgestellte Slicing-Hierarchie sind damit drei Ebenen im Blick (Bild 2):

- Im Gesamtsystem ist verortet, dass und wo Benutzer mit ihm umgehen. Das geschieht immer durch einen Adapter, der Portal genannt wird. Dieses Portal kapselt die Details von Sammlung und Projektion. Es ist strikt vom Code zu trennen, der die eigentliche Arbeit der Transformation von Input in Output übernimmt.
- Jede Input-Output-Wandlung findet im Rahmen einer Interaktion statt, die innerhalb eines Portals verortet ist.
- Und jede Interaktion triggert eine Funktion, den Entry Point für die Verarbeitung des Inputs, der in Form eines Requests angeliefert wird.

Die drei Ebenen sind in jedem Softwaresystem im Spiel. In jeder Anforderungsanalyse ist es mithin zentral, Folgendes zu identifizieren:

- Wer sind die Benutzer des Systems, und über welche Art von Frontend wollen sie mit ihm interagieren? Die Antwort besteht aus einer Anzahl von Portalen mit zugehörigen Technologien.
- Welche Interaktionen sollen zwischen Benutzern und System ablaufen? Wie wollen User „das System reizen“, um ihm das gewünschte Verhalten zu entlocken? Die Antwort besteht aus einer Anzahl von technologiespezifischen Frontend-Elementen.
- Welche Form soll die in jeder Interaktion getriggerte Funktion haben? Die Antwort besteht mindestens aus einer ►

Das gesamte
Software-
system im Blick
zu Beginn der
Outside-in-An-
forderungsanalyse
entlang der Slicing-
Hierarchie (Bild 3)



Funktionssignatur. Um automatisierte Tests dort ansetzen zu können, muss allerdings auch geklärt sein, welcher Zustand wie von der Funktion gebraucht wird. Außerdem sind dazu passende Testfälle zu sammeln. Hier geht es um die Entry Points.

Interaktionen

Auch wenn eine Hierarchie die Entwicklung des Verständnisses von einem Ende her nahelegt – entweder top-down oder bottom-up –, möchte ich für die Erkundung der Slicing-Hierarchie anders vorgehen.

Die oberste Ebene des Big Picture hat der vorherige Artikel [1] beschrieben: Dort oben geht es ums ganze Softwaresystem (Bild 3).

- Im Sinne einer Outside-in-Analyse soll zunächst geklärt werden, welcher Kontaktpunkte mit der Umwelt es bedarf. Denn jedes Softwaresystem hat nur einen Zweck in Bezug auf seine Umwelt, das heißt für seine User. Ihr Bedarf treibt Verhalten und Struktur im Inneren des Softwaresystems. Immer. Deshalb Outside-in-Analyse.
- Darüber hinaus kann jedes Softwaresystem diesen Bedarf nur erfüllen, wenn es Zugriff auf alle nötigen Ressourcen in der Umwelt hat. Diese Abhängigkeiten müssen also auch von Anfang gesammelt werden.

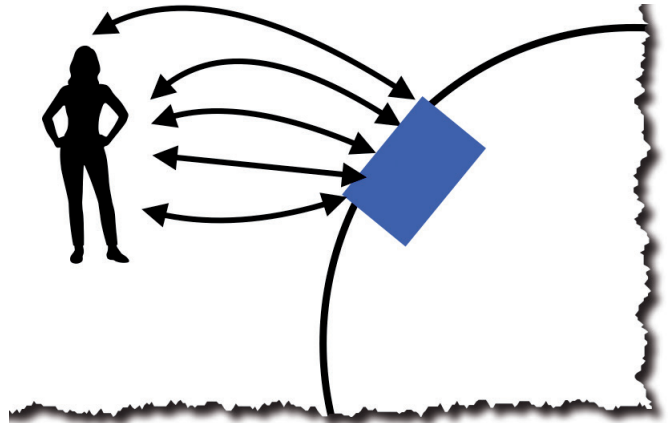
Wie es nach einer solchen initialen Analyse weitergeht, hängt allerdings von der Komplexität eines Softwaresystems ab. Kann nur ein kleiner Schritt gemacht werden, weil die Anforderungen so umfangreich sind? Oder kann in der Hierarchie tiefer gesprungen werden, weil schon Klarheit herrscht?

Für mich ist ein Kriterium für die Sprungtiefe, inwiefern ich wichtige beziehungsweise interessante Entry Points erkennen kann. Liegen die auf der Hand, muss ich mich nicht zwangsläufig durch Context, App und so weiter nach unten graben. Aus diesem Grund springe ich auch mit der Beschreibung der Slicing-Hierarchie nun über einige Ebenen hinweg zu den Interaktionen. Zu denen haben Sie sofort einen Bezug; damit kann ich Sie hoffentlich auf den Haken nehmen für eine spätere Reise zu höherliegenden Ebenen.

Interaktionen sind das sichtbare und technologische Pendant zu Entry Points. Wo ein Entry Point immer nur eine Funktion darstellt, die nur Sie als Entwickler interessiert, ist eine Interaktion etwas, das für den Benutzer unmittelbar relevant ist. Er kann sie sehen und spüren. Ein Mausklick, ein Tastendruck, eine Geste kann eine Interaktion starten, die durch eine Reaktion der Software mit einer Ausgabe in Form von Text, Bild, Ton oder auch noch anderen Modalitäten endet.

Die unmittelbar einleuchtende Frage an POs, Benutzer und Kunden während der Anforderungsanalyse ist deshalb: „Welche Interaktionen wünscht ihr euch denn?“

Leider ist die Antwort auf diese Frage meistens unmöglich. Interaktionen in nicht trivialer Software sind einfach so zahlreich. Sie gehen in die Hunderte, gar Tausende (Bild 4). Sie lassen sich nicht aus dem Stand aufzählen. Selbst ein kleines Beispiel macht schon deutlich, wie schwierig es ist, sie zu identifizieren. Als Szenario mag eine Aufgabenverwaltung dienen:



Benutzer wollen gewöhnlich in vielerlei Weise mit einer Software durch ein Portal interagieren. Welche Interaktionen nötig sind, können sie allerdings nicht einfach aufzählen (Bild 4)

- Aufgaben sollen in Listen angelegt werden können.
- Wenn eine Aufgabe überfällig ist, soll daran erinnert werden.

Zählen Sie jetzt einfach nur die Interaktionen auf, die Ihnen dazu einfallen. Eine Interaktion ist eine Antwort auf die Frage: Wann soll was passieren?

Welche Interaktionen stehen in Ihrer Liste? Ich komme nach der sehr knappen Anforderungsbeschreibung mindestens auf diese:

- Liste einrichten
- Liste umbenennen
- Liste löschen
- Listen anzeigen
- Liste auswählen, um sie mit ihren Aufgaben anzuzeigen
- Aufgabe einrichten in einer Liste
- Aufgabe bearbeiten
- Aufgabe löschen
- Benachrichtigen über überfällige Aufgabe

Das sind vor allem grundlegende CRUD-Interaktionen. Nicht spannend, da steckt keine Kreativität drin – dennoch darf keine vergessen werden und der Code dazu muss fehlerfrei sein.

Aber es gibt noch weitere Interaktionen:

- Programm starten
- Aufgabe in einer Liste verschieben
- Liste in der Liste der Listen verschieben
- Programm beenden

Was noch? Ist Ihnen noch etwas eingefallen? Nein? Das bedeutet nicht, dass es nicht weitere Interaktionen gibt. Welche das sind, kann nur ein Gespräch mit dem Kunden oder PO ergeben.

Worin unterscheidet sich das von der üblichen Anforderungsanalyse? Im Ziel! Ziel des Gesprächs ist immer eine Liste von Triggerfunktionen, denn diese stehen hinter den Interaktionen. Deshalb sollte nach meiner Ansicht jede Anforderungsanalyse das Ziel haben, dem PO die Liste der Interaktionen zu entlocken. Auf welchem Weg auch immer.

Hier sind typische Fragen, mit deren Hilfe Sie das Gespräch leiten können:

- „Welche Buttons möchtest du klicken können?“
- „Welche Menüpunkte brauchst du?“
- „Sollen Tastendrücke bei Eingabe der Daten das Verhalten auslösen?“
- „Wann und wie soll Verhalten X eigentlich ausgelöst werden?“
- „Kommt es zu Verhalten Y durch einen ‚Reiz‘ des Anwenders oder automatisch?“

Alle laufen darauf hinaus, dass der PO ganz konkret werden muss. Das ist wichtig! Nur wenn er konkret wird, kann es Klarheit für Sie geben. Je umfangreicher die Anforderungen, desto weniger kann ein PO allerdings dazu früh in der Analyse sagen. Deshalb die Slicing-Ebenen oberhalb der Interaktionen. Doch dazu später mehr.

Manchmal ist die Entscheidung, ob ein Verhalten durch Klick auf einen Button oder Menüpunkt ausgelöst werden soll, natürlich nicht so wichtig. Vor allem geht es darum, die Auslösung des einen Verhaltens vom anderen zu unterscheiden. Insofern ist es auch nicht immer wichtig, sofort die finale Benutzerschnittstelle zu kennen. Es kann auch mit einer prototypischen Benutzerschnittstelle angefangen werden.

Wieder das Beispiel Aufgabenverwaltung: Nach Klärung der Interaktionen wird das Verhalten implementiert, doch die Benutzerschnittstelle ist lediglich in der Konsole. Dort ist sie sehr einfach und ohne spezielle UI-Technologiekennntnisse zu implementieren. Die Interaktionen werden durch Kommandoeingaben angestoßen (Bild 5). Mit einem solchen Frontend-Prototyp fällt die Konzentration auf das Wesentliche leichter: die Domänenlogik. Sie steht hinter den Triggerfunktionen.

Später kann das Frontend – Sammlung und Projektion – ausgetauscht werden, ohne den getesteten Code hinter den Triggerfunktionen anfassen zu müssen. Es kann am Produktionsfrontend auch parallel zur Verhaltenszeugung durch

```
$ deno run aufgabenverwaltung.ts
--- Aufgabenverwaltung ---
? lc Liste 1
1. Liste 1
? lc Liste 2
1. Liste 1
2. Liste 2
? ls 1
Liste 1
? tc Aufgabe 1
Liste 1
1. Aufgabe 1
? tc Einkaufen; Äpfel, Birnen, Butter
Liste 1
1. Aufgabe 1
2. Einkaufen
Äpfel, Birnen, Butter
?
```

Prototypische Benutzerschnittstelle für eine Aufgabenverwaltung (Bild 5)

```
void MyButton_click(...) {
    var request = Collect(...);
    var response = blackbox.Process(request);
    Project(response);
}
```

Ein Eventhandler ruft einen Entry Point (Bild 6)

die Triggerfunktionen gearbeitet werden, wenn die Entry Points präzise definiert sind. Das Vorgehen mit Slicing wirkt auf diese Weise produktivitätsfördernd: In Arbeitsteilung können unabhängige Bereiche unabhängig vorangetrieben werden.

Bei Webanwendungen passiert das in gewisser Weise selbstverständlich durch die Schnittstelle zwischen Frontend (HTML/JS) und zum Beispiel REST-Backend. Ich plädiere jedoch dafür, auch innerhalb eines solchen Frontends noch genauer hinzuschauen. Mehr dazu, wenn wir auf die Worker-Ebene der Slicing-Hierarchie schauen.

Ab einem gewissen Punkt wird das vereinfachte UI natürlich auch umständlich. Seien Sie sensibel für die Grenze zwischen Nutzen und Nachteil. Vor allem wollte ich dadurch klarmachen, welche Optionen es eröffnet, wenn die Anforderungsanalyse sich von vornherein auf die Herausarbeitung von Interaktionen konzentriert:

- Der PO wird zu Klarheit gezwungen.
- Ein beteiligter UX-Designer beziehungsweise UI-Technologiespezialist kann bei der Entscheidung helfen, damit optimale Interaktionen und dahinter Triggerfunktionen entstehen.
- Sie als Entwickler bekommen bestmögliche Startpunkte für Entwurf und Implementation von Code.

Ein weiterer zu berücksichtigender Punkt bei der Entwicklung der Interaktionen – denn nicht weniger ist es als eine Entwicklung; Interaktionen müssen aus den Vorstellungen über das Softwareverhalten herausgeschält werden – ist der Zustand des Frontends.

Hält das Frontend selbst Zustand, oder bekommt es immer alle nötigen Daten von den Triggerfunktionen? Schauen Sie Bild 5 erneut an. Aus der Systemanalyse ist klar geworden, dass es mindestens zwei Codeteile gibt, nämlich

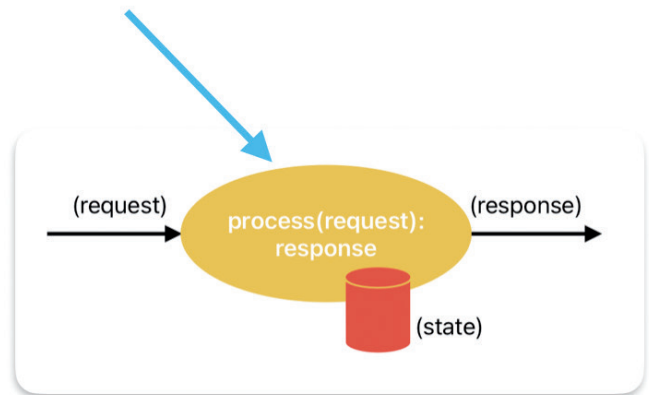
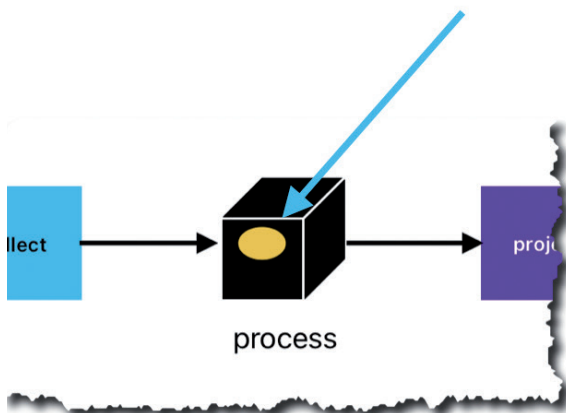
- das Frontend und
- den Rest, die Blackbox dahinter, die getriggert wird.

Nach dem Kommando `lc Liste 2` zeigt es die Liste der bisher angelegten Listen für Aufgaben an. Welcher der beiden Codeteile kennt diese Liste? Es ist anzunehmen, dass die Blackbox die Listen kennt und verwaltet, wahrscheinlich sogar persistent. Wie soll dann die Triggerfunktion aussehen? Zwei Varianten als Beispiel:

- `list_create(name:string):List`
- `list_create(name:string):List[]`

Eine Diskussion, ob `List` ein Domänenententyp ist oder nur einer für die Schnittstelle zwischen Frontend und Blackbox, erspare ich Ihnen an dieser Stelle. Mir geht es darum, ob nur ►

Triggerfunktion SOON Entry Point



Triggerfunktion und Entry Point sind dasselbe auf unterschiedlichen Ebenen der Slicing-Hierarchie (Bild 7)

die neu angelegte Liste oder alle Listen zurückgegeben werden sollen.

Wenn es immer alle sind, benötigt das Frontend keinen eigenen Zustand, um als Reaktion auf *lc* alle Listen aktualisiert zu zeigen. Wenn hingegen nur die neu angelegte zurückkommt, muss es sich zumindest die Namen aller anderen Listen gemerkt haben, um sie anzeigen zu können.

Ich will hier keiner der Varianten den Vorzug geben. Manchmal ist nur eine möglich, manchmal liegt eine nahe. Welche Variante ist für die eine Interaktion günstig, welche für die andere?

Wichtig ist vor allem, dass im Rahmen der Interaktionsentwicklung genau darüber gesprochen wird. Welche UI-Technologie soll genutzt werden, welche Herangehensweise unterstützt sie? Zumindest Sie als Entwickler müssen sich darüber Klarheit verschaffen. Davon hängt die Form der Triggerfunktion ab.

Entry Points

Interaktionen finden dort statt, wo Anwender Ihre Software „reizen“. Heutzutage geschieht das oft im Rahmen von Events, die von Eventhandlern bedient werden. Das ist seit Visual Basic 1.0 (VB) so.

Der Erfolg von VB ist aus meiner Sicht zu einem guten Teil darauf zurückzuführen, dass die Integration der Benutzerschnittstellengestaltung mit der Codierung des Verhaltens so eng war. Wer wusste, wie das UI aussah, hatte sofort eine simple Grundstruktur für den Code: Jedem Button, jedem Menüpunkt und so weiter stand sofort eine Funktion gegenüber, in die Logik „gekippt“ werden konnte, die das getriggerte Verhalten erzeugte. Das war der Hit! Dass das leider nicht automatisch zu sauberem Code führte, mussten Entwickler erst leidvoll lernen.

Deshalb muss ich noch einmal klarstellen: Mit Entry Points meine ich keine Eventhandler einer Benutzerschnittstellentechnologie! Eventhandler-Funktionen sind technologiespezifisch und gehören zum Portal als Adapter für die Verbindung zwischen der Umwelt und der Innenwelt des Softwaresystems.

Eventhandler lassen sich nicht gut testen. Bei einer echten Trennung von „Model“ wie bei WPF sieht es besser aus, dennoch rechne ich zum Beispiel ein dortiges Command immer noch zur UI-Technologie. Darin darf nicht die Musik spielen, die automatisiert getestet werden muss.

Triggerfunktionen sind mithin Funktionen, die getrennt von Eventhandlern existieren. Sie gehören „zum Rest“ der Software und könnten von Eventhandlern aufgerufen werden (auch wenn ich eine andere Anbindung vorziehe; doch das ist ein Detail).

Im Slicing sähe ein Eventhandler wie in Bild 6 aus: Er kümmert sich um die Sammlung von Input, um den Request zu schnüren. Er ruft die Triggerfunktion. Er kümmert sich um die Projektion des Response vom Entry Point.

Bei HTTP-Endpunkten sind Sammlung und Projektion oft trivial, weil sie von der Infrastruktur übernommen werden. Das können Sie dankend annehmen. Doch der Controller bleibt ein Adapter, der Verhalten nicht selbst erzeugt, sondern dafür den eigentlichen Entry Point aufruft.

Was unterscheidet den Begriff Triggerfunktion von Entry Point (Bild 7)? Es geht um dieselbe Funktion. Im Rahmen der Diskussion um die Interaktionen ist diese Funktion aber noch

```

1 export type List = {
2   id:string;
3   name:string;
4   description:string
5 }
6
7 export function list_create(name:string):List {}
8
9 export function list_rename(id:string, newName:string):List {}
10
11 export function list_delete(id:string) {}
12
13 export function lists_all():List[] {}
14
15 export function program_start():List[] {
16   return lists_all();
17 }

```

Die Entry Points eines Teils der Aufgabenverwaltung (Bild 8)


```

1 import * as sut from "../blackbox.ts";
2
3 import { assertEquals } from "https://deno.land/std@0.167.0/testing/asserts.ts";
4
5 Deno.test("scenario", () => {
6   // 1. Das Programm wird gestartet; noch gibt es keine Listen.
7   var lists = sut.program_start();
8   assertEquals(lists.length, 0);
9
10  // 2. Es wird eine Liste hinzugefügt.
11  var list = sut.list_create("L1");
12  assertEquals(list.name, "L1");
13  assertEquals(list.description, "");
14  const lid1 = list.id;
15
16  // 3. Die Listen werden alle abgerufen.
17  lists = sut.lists_all();
18  assertEquals(lists, [
19    {
20      id: lid1,
21      name: "L1",
22      description: "",
23    },
24  ]);
25
26  // 4. Es wird noch eine Liste hinzugefügt.
27  list = sut.list_create("L2");
28  const lid2 = list.id;
29
30  // 5. Die Listen werden wieder abgerufen.
31  lists = sut.lists_all();
32  assertEquals(lists, [
33    {
34      id: lid1,
35      name: "L1",
36      description: "",
37    },
38    {
39      id: lid2,
40      name: "L2",
41      description: "",
42    },
43  ]);
44
45  // 6. Eine Liste wird umbenannt.
46  list = sut.list_rename(lid1, "L1v2");
47  assertEquals(list.id, lid1);
48  assertEquals(list.name, "L1v2");
49
50  // 7. Eine andere Liste wird gelöscht.
51  sut.list_delete(lid2);
52
53  // 8. Die Listen werden wieder abgerufen.
54  lists = sut.lists_all();
55  assertEquals(lists, [
56    {
57      id: lid1,
58      name: "L1v2",
59      description: "",
60    },
61  ]);
62 });
63

```

Alle Entry Points

werden in einem
Szenariotest über-
prüft (Bild 9)

relativ schwammig. Vor allem ist wichtig zu verstehen, dass es sie gibt und mit welchen Daten sie grundsätzlich arbeitet:

- Es muss einen Request aus einer Input-Sammlung geben.
- Es wird ein Response geliefert.
- Es ist vielleicht Zustand/ein Seiteneffekt im Spiel.

Bei der Ebene des Entry Points darunter geht es um die Verfeinerung dieser Funktion: Wie genau sieht die Signatur aus? Welche Daten sollen in welcher Struktur in den Entry Point hineingegeben werden, welche kommen am besten heraus, welche braucht er woher darüber hinaus, und auf welche hat er welchen Einfluss? Erst wenn das geklärt ist, können auch Testfälle bestimmt werden.

Auf der Ebene der Entry Points wird es wirklich relevant, dass Sie sich Gedanken über die Zustandshaltung machen: Frontend oder Blackbox? Dies hat Einfluss auf die Signatur des Entry Points, wie oben gezeigt.

Ich tendiere dabei zu besserer Testbarkeit. Meine Frage ist: Wie kann ich mehr Code leichter testen? Code im Frontend ist tendenziell schwerer zu testen, also versuche ich ihn zu minimieren. Wenn also Zustandshaltung im Frontend Aufwand bedeutet, ziehe ich den Code lieber in die Blackbox, wo er unabhängig von Frontend-Technologie testbar ist.

Wenn die Zustandshaltung im Frontend hingegen trivial ist, deklarativ beziehungsweise intentional funktioniert und keinen (testwürdigen) Code braucht, dann lasse ich Zustand auch mal im Frontend. Das erspart mir Datenverkehr mit der Blackbox. Die Entry Points können sich mehr auf das Wesentliche konzentrieren. Wenn zum Beispiel die Listen einer Aufgabenverwaltung in einer GUI-Liste einer Desktop-Anwendung angezeigt werden sollen, erübrigt sich eine Zustandshaltung in der Blackbox jenseits der zu leistenden Persistenz.

Auch hier ist wieder zu sehen, wie UI/UX-Entscheidungen nach innen wirken. Die Blackbox, also den Kern eines Softwaresystems, gänzlich unabhängig von der Umwelt, von den Interaktionen, von den Frontend-Technologien zu halten, ist eine durchaus schädliche Optimierung. Natürlich soll die konkrete Nutzung von UI-Frameworks in Portalen gekapselt sein – doch deren Paradigma darf nach innen wirken.

Das muss allerdings eine bewusste Entscheidung sein. Denn je mehr die äußere Form nach innen wirkt, desto abhängiger ist das Innen natürlich. Die äußere Form zu wechseln mag dadurch erschwert werden. Hier ist eine Balance zwischen Effizienz und Flexibilität zu finden.

Oben habe ich eine Anzahl von möglichen Interaktionen einer Aufgabenverwaltung gelistet. Zu jeder gehört eine Triggerfunktion. Hier nochmal ein Auszug:

- Liste einrichten: *list_create()*
- Liste umbenennen: *list_rename()*
- Liste löschen: *list_delete()*
- Listen anzeigen: *lists_all()*
- Programm starten: *program_start()*

Das ist trivial und bedarf eigentlich keiner Dokumentation oder sogar Erwähnung. Das haben Sie immer im Hinterkopf.

Nur die letzte Interaktion lässt fragen, ob sich eine eigene Triggerfunktion lohnt oder vielleicht bei Programmstart *lists_all()* aufgerufen werden könnte. Um eine vorzeitige Optimierung zu vermeiden, würde ich allerdings dafür plädieren, zunächst mit *program_start()* zu planen. Vielleicht ergibt eine spätere Analyse, dass bei Programmstart noch mehr oder etwas anderes getan werden soll, als nur die Aufgabenlisten zu listen.

Für den Level „Entry Points“ in der Slicing-Hierarchie ist nun etwas mehr Detail gefragt: Wie soll denn die Signatur ganz präzise aussehen? Ich zeige Ihnen einfach meine Entscheidungen und verzichte auf eine Diskussion, warum sie so ausgefallen sind. Es geht hier nicht darum, die absolut beste Signatur zu erarbeiten, sondern den Unterschied zwischen den Levels bei Slicing zu verdeutlichen. ►

Bild 8 zeigt die Entry Points für die obigen Triggerfunktionen, wie ich sie mir beispielhaft vorstelle. Sie sehen, ich habe mich dafür entschieden, das Frontend durchaus Zustand halten zu lassen.

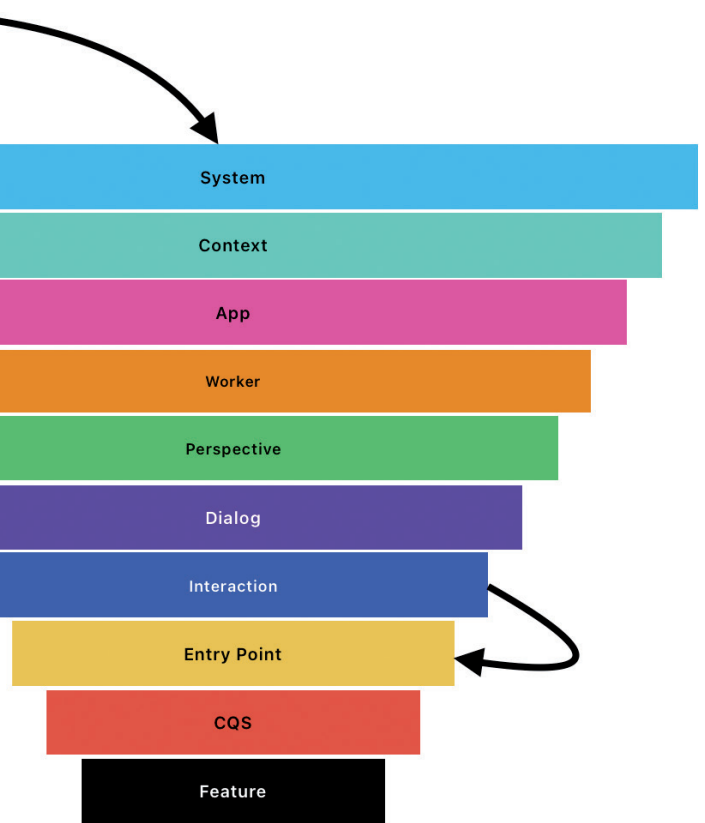
Jetzt sind alle Funktionen mit Parametern und Typen versehen. Zustand ist implizit. Die Sprache ist TypeScript. Eine Klasse habe ich für die Blackbox nicht angelegt. Die Funktionen werden einfach von einem dateibasierten Modul exportiert. Ob das die beste Idee ist, lasse ich dahingestellt. Es soll nicht um die Feinheiten des Entwurfs und der Modularisierung gehen, sondern darum, dass die Entry Points eben eine Schnittstelle darstellen, die dem Frontend bekannt ist. Die Entry Points isolieren die Blackbox von der Umsetzung der Interaktionen mit dem Benutzer.

Zu den konkreten Schnittstellen gehören natürlich auch Datentypen. *List* als Struktur für eine Aufgabenliste ist Teil der Schnittstelle, die die Entry Points definieren. Und was ist mit den persistenten Daten?

Die persistenten Daten sind nicht Teil der Schnittstelle. Von ihnen eine Idee zu haben ist nicht falsch. Sicher werden sie in diesem Fall nah an *List* sein. Doch ich denke, für die Formulierung von Testfällen, die ja auch noch fehlen, sind die persistenten Daten gar nicht so wichtig. Überhaupt ist Persistenz nicht entscheidend, um ein Frontend an eine Blackbox mit dieser Oberfläche zu binden. Sie muss nur zusichern, den Zustand korrekt zu bewahren.

Jeder Entry Point könnte einzeln getestet werden. In diesem Fall glaube ich jedoch, dass ein „Szenariotest“ einfacher ist. So nenne ich einen Test, der mehrere Funktionen im Verein auf die Probe stellt. Ein Szenariotest simuliert eine Folge von Interaktionen, zum Beispiel

- Das Programm wird gestartet; noch gibt es keine Listen.
- Es wird eine Liste hinzugefügt.



Sprünge sind in der Slicing-Hierarchie erlaubt, sobald Klarheit über die Zerlegung einer Ebene herrscht (Bild 10)

- Die Listen werden alle abgerufen.
- Es wird noch eine Liste hinzugefügt.
- Die Listen werden wieder abgerufen.
- Eine Liste wird umbenannt.
- Eine andere Liste wird gelöscht.
- Die Listen werden wieder abgerufen.

Zu jedem Schritt lassen sich Erwartungen formulieren. Bild 9 zeigt alle Entry Points in einem solchen Zusammenspiel. Dass

```

9 export function list_create(name:string):List {
10   const newList = {
11     id: (_lists.length + 1).toString(),
12     name: name,
13     description: ""
14   }
15   _lists.push(newList);
16   return newList;
17 }
18
19 export function list_rename(id:string, newName:string):List {
20   const list = _lists.find((l) => l.id === id);
21   if (list === undefined) throw new Error("Rename: List not found!");
22
23   list.name = newName;
24   return list;
25 }
26
27 export function list_delete(id: string) {
28   const index = _lists.findIndex((l) => l.id === id);
29   if (index !== -1) {
30     _lists.splice(index, 1);
31   }
32 }

```

Potenzial für Refaktorisierung im Sinne von CQS (Bild 12)

```

15 blackbox.ts x +
16 blackbox.ts > ...
17
18 1 export type List = {
19   2   id:string;
20   3   name:string;
21   4   description:string
22   5 }
23
24 6
25 7 const _lists:List[] = [];
26
27 8
28 9 export function list_create(name:string):List { ... }
29
30 10
31 11 export function list_rename(id:string, newName:string):List { ... }
32
33 12
34 13 export function list_delete(id: string) { ... }
35
36 14
37 15 export function lists_all():List[] { ... }
38
39 16
40 17 export function program_start():List[] { ... }

```

Nicht alle Entry Points sind nur Command oder nur Query (Bild 11)

die Implementation dahinter derzeit nur In-Memory Zustand hält, ist für das Prinzip, um das es mir hier geht, unerheblich.

Die Anforderungen des Beispiels haben es erlaubt, schnell auf eine ganze Reihe von Interaktionen mit ihren Entry Points zu kommen. Manchmal ist das möglich. In anderen Fällen ergeben sich für zum Beispiel eine User Story mehrere Interaktionen, doch die können nicht alle näher betrachtet werden (breadth-first), sondern auf eine der Interaktionen ist besonders die Aufmerksamkeit zu lenken (depth-first), um für sie Signatur, Zustand/Seiteneffekte und Testfälle zu definieren.

Wie sehr sie in die Breite gehen, ist auch für die Methode unerheblich. Letztlich sollen Sie bei einem Entry Point ankommen, dessen Ausgestaltung Ihnen erlaubt, dahinter mit Entwurf und Codierung zu beginnen. Und dann ist der nächste Entry Point dran, und danach der nächste und so weiter.

Der Weg zu den Entry Points für die kleine Beispielsoftware bestand aus drei Sprüngen in der Slicing-Hierarchie (Bild 10):

- Der Einsprung erfolgt für die System-Ebene. Dort geht es immer los: Überblick gewinnen, Frontends identifizieren.
- Da das Beispiel so klein ist, sind wir sofort zur Interaktion-Ebene gesprungen. Wir konnten alle Interaktionen ohne weiteres aufzählen.
- Für jede Triggerfunktion hinter den Interaktionen haben wir dann den Sprung zur Entry-Point-Ebene gemacht.

Sprünge über Ebenen hinweg sind okay, wenn das, was auf der nächsten Ebene gesucht wird, auf der Hand liegt. Das war bei den Interaktionen für die Aufgabenverwaltung klar. Zu den Entry Points weiterzugehen war der nächste nötige Schritt.

Was aber nun? Können wir schon mit den Entry Points zufrieden sein? Wenn die dahinterstehende Funktionalität überschaubar ist, ja. Dann einfach umsetzen. Und was, wenn sie immer noch unhandlich groß ist? Dann gilt es, eine Ebene tiefer zu steigen.

Command Query Separation (CQS)

Wie können Entry Points formal (!) weiter zerlegt werden? Was wäre für Sie als Entwickler ein relevanter struktureller (!) Schnitt? Ich denke, das Prinzip der Command Query Separation von Martin Fowler [2] ist hier hilfreich. Mit ihm kann Funktionalität zumindest auf einen Aspekt der Zustandsveränderung fokussiert werden:

- Command: Eine Funktion verändert Zustand und liefert kein Resultat zurück. Oder wenn ein Resultat geliefert wird, dann sind es Metadaten zur Aktivität, zum Beispiel die Zahl der gelöschten Datensätze oder die ID eines neu angelegten Datensatzes. Bei REST entspricht dem ein POST-Request. Ein Command kann inhaltlich fehlschlagen, weil zu verändernde Daten nicht im erwarteten Zustand sind.
- Query: Eine Funktion verändert keinen Zustand und liefert ein Resultat zurück, das auch auf Zustand basieren kann. Dem entspricht bei REST ein GET-Request. Queries können inhaltlich nicht fehlschlagen.

Die Unterscheidung von Veränderungen und Abfragen hat mehrere Vorteile: ►



HANDS-ON-WORKSHOPS
UND WEITERBILDUNG FÜR
SOFTWARE-ENTWICKLER UND
-ARCHITEKTEN

OFFENE
TRAININGS!

Unsere öffentliche Hands-on Workshops sind wieder da.

In 2024 können Sie folgende Workshops buchen

C# für Fortgeschrittene

27.-29.05.2024

UX und UI-Design
für Entwickler

11.-12.06.2024

Clean Code und Software Design

28.-30.10.2024

Architekturen für
.NET-Anwendungen

11.-13.11.2024

In Vorbereitung

C#- Grundlagen

3 Tage

Azure DevOps - CI/CD mit
Azure Pipelines und Git

3 Tage

Erste Schritte mit Docker
für .NET-Entwickler

2 Tage

developer-academy.de

Ihre Ansprechpartnerin: Susanne Herl

+49 731 88005-8835 • susanne.herl@ebnermedia.de

- Getrennte Testbarkeit dieser unterschiedlichen Weisen des Umgangs mit Daten.
- Technologische Trennung dieser Modi, zum Beispiel kann ein Command Locks erfordern, Queries aber nicht.
- Wiederverwendbarkeit in unterschiedlichen Zusammenhängen wegen feingranularerer Funktionen.

Durch diese Brille betrachtet sind Entry Points oft nicht eindeutig. Sie bieten Potenzial für eine Verfeinerung, das heißt Verkleinerung des Scopes durch Konzentration auf Command versus Query.

Wie steht es in dieser Hinsicht mit den bisherigen Entry Points der Aufgabenverwaltung? Bild 11 zeigt, dass das Urteil nur in drei von fünf Fällen eindeutig ist:

- `list_delete()` ist ein Command, da Zustand verändert wird – eine Liste wird gelöscht –, aber keine Daten zurückgeliefert werden. Die Annahme ist, dass die per `id` identifizierte Liste existiert.
- `lists_all()` und `program_start()` sind beide Queries. Sie fragen lediglich die Liste der Listen ab und liefern Einträge zurück.

Sowohl `list_create()` wie auch `list_rename()` hingegen sind keine reinen Kommandos, auch wenn das durch die Benennung suggeriert wird. Der Widerspruch zum CQS besteht in ihren Responses, die die veränderten Daten zurückliefern. Das ist naheliegend, jedoch streng genommen nicht erlaubt. Wie reagieren Sie darauf?

Das Wichtigste ist, diesen Widerspruch überhaupt entdeckt zu haben. Jetzt können Sie eine bewusste Entscheidung treffen, ob sie diese beiden Aspekte gekoppelt lassen oder sie aufdröseln. In manchen Fällen kann es bleiben, wie es ist, in anderen lohnt sich eine Trennung.

Insbesondere, wenn die zurückzuliefernden Daten schon bei der Kommandoausführung ganz natürlich „anfallen“ und nicht zu umfangreich sind, wäre es dumm, sie nicht gleich zurückzuliefern. Das erspart dem Aufrufer einen zweiten Zugriff, um sie zu beschaffen. Aber, wie gesagt, es kommt auf den Einzelfall an.

Hier ist jedenfalls Potenzial für feinere Schnitte. Wie könnten diese aussehen? Ein Blick hinter die Kulissen der Imple-

```
export function list_rename_conv(id:string, newName:string): List {
    list_rename(id, newName);
    return list_get(id);
}
```

Ein Convenience Entry Point als Integration anderer Entry Points (Bild 14)



Nach Refaktorisierung entsprechen alle Entry Points dem CQS (Bild 13)

mentation, die ich schon (vorschnell) vorgenommen hatte, um die Szenariotests oben vorzustellen, zeigt Potenzial für die Trennung: Es gibt nicht nur einen Widerspruch zu CQS, sondern auch zum Prinzip Don't Repeat Yourself (DRY) (Bild 12). In zwei Funktionen wird eine Liste über ihre ID herausgesucht.

Wenn diese Logik in eine eigene Funktion wandert, kann dem CQS Genüge getan werden (Bild 13):

- `list_rename()` und `list_delete()` können sich besser auf ihre eigentliche Aufgabe konzentrieren.
- Aufrufer von `list_create()` und `list_rename()` können sich die bisher zurückgelieferte Liste über einen weiteren Entry Point `list_get()` beschaffen.

Alle Entry Points entsprechen nun dem CQS:

- `list_delete()` hat sich am wenigsten verändert. Die Funktion sucht nur einfach nicht mehr selbst die zu löschende Liste heraus.
- `list_rename()` delegiert die Beschaffung der umzubenennenden Liste nun auch – liefert sie allerdings nicht mehr zurück. Die Funktion ist damit zu einem klaren Command geworden.
- `list_create()` retourniert jetzt nicht mehr die neue Liste, sondern nur deren ID. Auch hier liegt jetzt ein reinrassiges Command vor.

- `list_get()` ist der neue Query Entry Point zur Beschaffung einer einzelnen Liste. Er kapselt die interne Methode, die noch sowohl Liste als auch Index liefert. Auf diese Weise konnten die Bedürfnisse beim Umbenennen und Löschen mit einer Funktion befriedigt werden.

Im Szenariotest ist diese Beschaffung natürlich hinzuzufügen (Zeile 12 im rechten Ausschnitt in **Bild 13**).

Aber wird dadurch nicht auch etwas mehr Klarheit hergestellt?

Solange nicht deutlich spürbare Performance-Verschlechterungen dagegen sprechen, ziehe ich ein Slicing nach CQS unterhalb der Entry Points vor. Ich möchte jeden Entry Point so dünn geschnitten haben, wie es möglich ist.

Allerdings... ich mag auch die Bequemlichkeit, die ein hybrider Entry Point wie zum Beispiel das ursprüngliche `list_rename()` geboten hat. Deshalb finde ich es okay, wenn Sie ganz bewusst nach einer Verfeinerung entlang des CQS die entstandenen Entry Points zu einem Convenience Entry Point zusammenfassen (**Bild 14**). In dem kommen Command und Query (auch mehrere) zusammen, um eine abstraktere Funktion zu bilden. Die muss auch eigentlich nicht mehr separat getestet werden, falls die darin vereinten Entry Points separat getestet wurden; dass Ihnen bei der Integration von zwei oder drei Entry Points ein Fehler unterläuft, ist kaum zu erwarten.

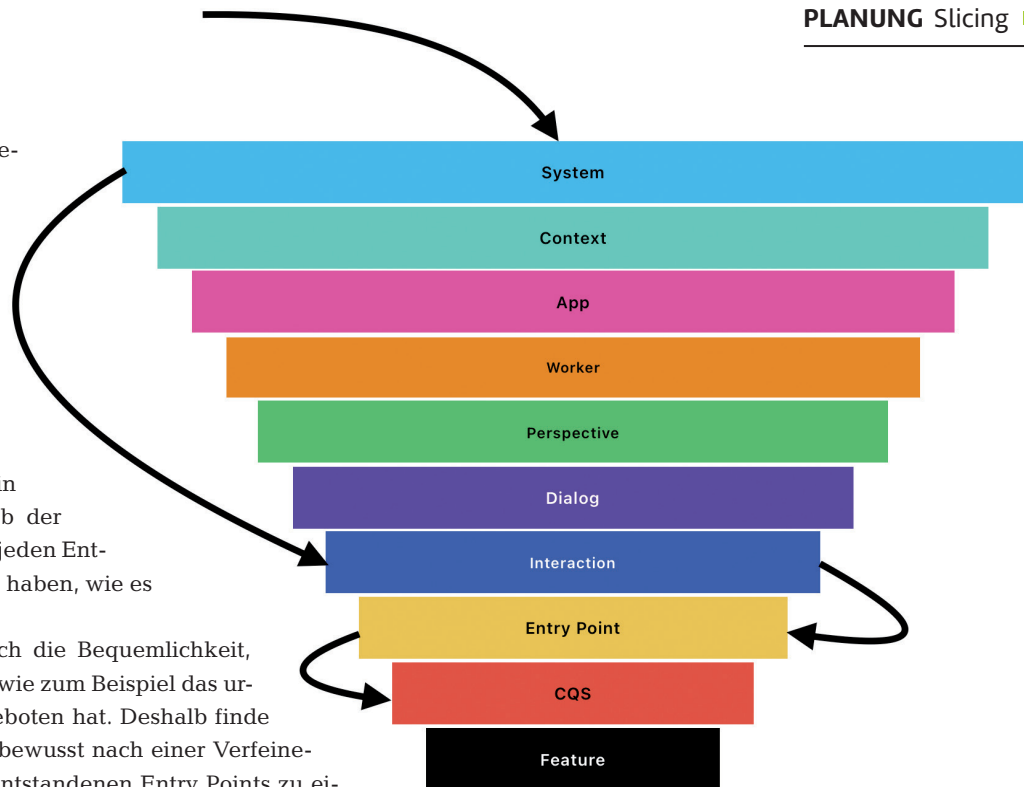
Zwischenstand

Slicing leitet in kleinen Schritten durch eine für Sie als Entwickler relevante Hierarchie von Inkrementen. Alles beginnt immer beim kompletten Softwaresystem – doch das Ziel ist am Ende die einzelne CQS-fokussierte, testbare Funktion (**Bild 15**), die unabhängig von Frontend-Technologien ist. Hinter ihr steht die Erzeugung von Verhalten mit einem beliebig tiefen Baum weiterer Funktionen, die die Logik enthalten, um gegebenenfalls unter Rückgriff auf Ressourcen die nötigen Datentransformationen durchzuführen, die der User wünscht.

Der Sprung vom Softwaresystem zu einer tieferen Ebene ist jederzeit möglich, wenn Sie fühlen, dass Sie auf der Zielebene Klarheit haben. Slicing ist ein Tool, um Ansatzpunkte für Ihren Softwareentwurf zu identifizieren, damit Sie möglichst geradlinig zu lauffähigem, korrektem Code kommen. Die Gestaltung der Benutzerschnittstelle ist dabei lediglich ein Hilfsmittel, falls die Entry Points für Sie noch nicht auf der Hand liegen. Deshalb hat sie in diesem Artikel noch keine Rolle gespielt.

Wenn Sie mit einer User Story oder einem Use Case konfrontiert sind, fragen Sie sich stets: „Weiß ich schon genug, um einen Entry Point zu formulieren?“ Wenn nein, bohren Sie hinein in die User Story. Brechen Sie den Use Case herunter. „Grillen“ Sie den PO. Lassen Sie ihn nicht ziehen, bevor Ihnen nicht die konkreten Interaktionen sonnenklar sind, in denen User das Softwaresystem triggern wollen.

Interaktionen sind das, was Anwender wollen. Sie wollen das Softwaresystem so kontrollieren, dass es tut, was sie sich



CQS-konforme Funktionen als Kern der Benutzerinteraktionen sind das Ziel immer feinerer Schnitte durch die Anforderungen entlang der Slicing-Hierarchie (**Bild 15**)

wünschen. Aber nicht nur, dass Software etwas Bestimmtes tun soll, sondern wie das konkret angestoßen werden soll, ist entscheidend. Die Granularität der Interaktionen ist bestimmend für die Form der Entry-Point-Funktionen.

Bei der Anforderungsanalyse werden Sie deshalb den größten Teil der Zeit auf den Slicing-Ebenen ab *Interaction* verbringen. Insbesondere das Aushandeln der Testfälle benötigt viel Zeit. POs legen sich ungern fest.

Um es Ihnen und POs in der Hinsicht einfacher zu machen, können Sie allerdings noch eine Ebene tiefer steigen als CQS. Features sind Inkremente innerhalb derer für Entry Points; die Funktionen sind definiert, dennoch kann der Scope feiner geschnitten werden. Das soll im nächsten Artikel geschehen. ■

[1] Ralf Westphal, *Den Stier bei den Hörnern packen*, dotnetpro 5/2024, Seite 26 ff., www.dotnetpro.de/A2405Slicing

[2] Martin Fowler, *Command Query Separation*, www.dotnetpro.de/SL2406Slicing1



Ralf Westphal

ist Trainer, Berater und Mitgründer der Clean Code Developer Initiative (<https://clean-code-developer.de>). Seine Schwerpunkte sind dauerhaft hohe Produktivität für die Softwareentwicklung und zukunftsfähige Teamorganisation.

<https://ralfw.de>

dnpCode

A2406Slicing